

Warm up

How is Assignment 5 going?

Precept 21: Assignment 6 overview and the “B” attack

Polly Ren

COS 217: Introduction to Programming Systems
Spring 2026
Princeton University

April 16, 2026

Overview

- 1 Assignment 6 overview
- 2 Creating the input file
- 3 Activity: analysing grader
- 4 Summary

grader.c

- Compiled with:
`gcc217 -O -fomit-frame-pointer grader.c -o grader`
 - `-O` (capital “oh”): the compiler may make optimisations that differ from what you would expect
 - ◇ Make sure to read the assembly code; do not assume the contents of memory or the stack frame
 - `-fomit-frame-pointer`: the compiler does not use a frame pointer register (e.g., `x29`)
- The program reads a string from stdin into a global `name` buffer in the bss section
- `mprotect((void*)((unsigned long)name & 0xffffffffffff000), 1, PROT_READ | PROT_WRITE | PROT_EXEC);`
 - Change the permissions of the memory page containing `name` (in bss) to allow execution
 - This allows us to execute code that we write into `name` (e.g., shellcode)
 - Program is intentionally and artificially susceptible to an attack
 - “B” attack would work without this, but “A” attack would not

Constructing a memory map

- Create a map of the memory layout of the program, including the text, data, bss and stack segments
- gdb has a disassembler, i.e., can look at the assembly code of the assembled and linked program
- Use x and p commands to examine memory layout of the program
 - `x/67i readString`: look at instructions in `readString`
 - `print &grade`: look at the address of `grade`
 - `x/x &grade`: look at the contents of `grade`
 - `print &name`: look at the address of `name`
- Memory map of the stack
 - Each function (`main`, `getName`, `readString`) has its own stack frame
 - `readString`'s stack frame
 - ◇ saved value of `x30` (where to return to in `getName`)
 - ◇ other callee-saved registers
 - ◇ the `buf` local array
 - ◇ why are `i` and `c` not stored on the stack even though they are local variables? → compiler optimisation uses registers to store them instead

The “B” attack

- Goal: provide a string input that causes the program to give you a “B” grade (as opposed to the default “D”)
- Observe: `readString` keeps reading from `stdin` until it reads EOF or newline
 - It does not check if the input exceeds the size of `buf`
- Insight: write beyond the end of the `buf` array to overwrite the saved value of `x30` in `getName`’s stack frame
- Where should we overwrite `x30` to point to?
 - Can we jump to the assignment of `grade = 'B'`?
- Approach: use the memory map to determine:
 - the address of the instruction that assigns `grade = 'B'`
 - how many bytes to write in order to overwrite `x30`

Overview

- 1 Assignment 6 overview
- 2 Creating the input file**
- 3 Activity: analysing grader
- 4 Summary

How should we create dataB?

- What are possible options for creating dataB?
 - Use emacs or a text editor to type the content
 - Limitations:
 - ◊ how to type non-printable characters (e.g., '0')?
 - ◊ how to type a memory address (e.g., 0x400000)?
- Solution: write a C program that writes the content of dataB

```
gcc217 createdataB.c -o createdataB
./createdataB
./grader < dataB
```

- Printable characters can be printed directly with `fprintf`

Writing non-printable characters

- Interestingly, non-printable characters can also be printed with `fprintf` using the `%c` format specifier
- Example: write five zero bytes to a file

```
FILE *psFile;  
psFile = fopen("data", "w");  
  
for (int i = 0; i < 5; i++) {  
    fprintf(psFile, "%c", '\\0');  
    /* fprintf(psFile, "%c", 0); */  
    /* fprintf(psFile, "%c", 0x00); */  
    /* putc('\\0', psFile); */  
    /* putc(0, psFile); */  
    /* putc(0x00, psFile); */  
}  
  
fclose(psFile);
```

Writing memory addresses

- Memory addresses can also be printed with `fprintf` using the `%c` format specifier, but we need to be careful about endianness
 - ARMv8 is little-endian → least significant byte is at lowest address
 - Need to write the bytes of the address in reverse order
- Example: write the 8-byte address `0x0123456789abcdef`

```
FILE *psFile;  
psFile = fopen("data", "w");  
  
fprintf(psFile, "%c", 0xef);  
fprintf(psFile, "%c", 0xcd);  
fprintf(psFile, "%c", 0xab);  
fprintf(psFile, "%c", 0x89);  
fprintf(psFile, "%c", 0x67);  
fprintf(psFile, "%c", 0x45);  
fprintf(psFile, "%c", 0x23);  
fprintf(psFile, "%c", 0x01);  
  
fclose(psFile);
```

Writing memory addresses using fwrite

Alternatively:

```
FILE *psFile;  
psFile = fopen("data", "w");  
  
unsigned long ulData;  
ulData = 0x0123456789abcdef;  
fwrite(&ulData, sizeof(unsigned long), 1, psFile);  
  
fclose(psFile);
```

- fwrite dumps the exact byte representation of ulData from memory into the file, with no conversion or formatting
- This write is endianness-dependent
 - On a little-endian machine, the bytes of ulData will be written in reverse order (i.e., 0xef first, then 0xcd, etc.)
 - On a big-endian machine, the bytes of ulData will be written in the same order as they appear in the code (i.e., 0x01 first, then 0x23, etc.)
- Can use a similar approach to write instructions

Overview

- 1 Assignment 6 overview
- 2 Creating the input file
- 3 Activity: analysing grader
- 4 Summary

Activity: analysing grader

- What is the instruction that gets executed after returning from `getName`?
- We can use `gdb` to confirm this by looking at what the value of `x30` is inside the function `getName`. How?
- What does `adrp` and `add` do?

Activity solutions

- What is the instruction that gets executed after returning from getName?
 - the instruction after the bl getName instruction:
0x40087c: adrp x1, 0x400000
- We can use gdb to confirm this by looking at what the value of x30 is inside the function getName. How?
 - 0x40087c is the value of x30 in getName, so x/i 0x40087c will show us the instruction at that address
- What does adrp and add do?
 - adrp x1, 0x400000: loads the page address into x1
 - add x1, x1, #0x910: adds the offset to get the final address
 - $0x400000 + 0x910 = 0x400910 \rightarrow$
x/s 0x400910: "Andrew_Appel"

Overview

- 1 Assignment 6 overview
- 2 Creating the input file
- 3 Activity: analysing grader
- 4 Summary

Summary

- Assignment 6 overview: constructing a memory map, the “B” attack, writing non-printable characters to a file
- Dates to remember:
 - Assignment 5: due Tue, 4/21 at 9pm
 - Assignment 5 Quiz: Wed, 4/22 in lecture
 - Assignment 6: due Wed, 4/29 at 9pm
- Acknowledgements
 - Most of the code in these slides is adapted from the precept materials, written by Prof. Bob Dondero
 - Can be accessed at <https://www.cs.princeton.edu/courses/archive/spring26/cos217/precepts/w12p2/>